TOPOLOGICAL SORT (Kahn's Algorithm, BFS-based)

Use when: ordering tasks with dependencies (DAG only — no cycles).

```
vector<int> indeg(n, 0);
for (int u=0;u<n;u++) for (int v : adj[u]) indeg[v]++;
queue<int> q; for (int i=0;i<n;i++) if (indeg[i]==0) q.push(i);
vector<int> order;
while (!q.empty()) {
    int u = q.front(); q.pop(); order.push_back(u);
    for (int v : adj[u]) if (--indeg[v] == 0) q.push(v);
}
// if order.size() < n → graph has a cycle, no valid ordering exists
```

DSU (Disjoint Set Union / Union-Find)

Use when: grouping elements, checking connectivity — nearly O(1) per operation.

```
vector<int> parent, rnk;
void dsuInit(int n) { parent.resize(n); iota(parent.begin(),parent.end(),0);
rnk.assign(n,0); }
int find(int x) { return parent[x]==x ? x : parent[x]=find(parent[x]); }  // path
compression
void unite(int a, int b) {
    a = find(a); b = find(b); if (a == b) return;
    if (rnk[a] < rnk[b]) swap(a, b);
    parent[b] = a; if (rnk[a] == rnk[b]) rnk[a]++;  // union by rank
}
bool connected(int a, int b) { return find(a) == find(b); }
```

DIJKSTRA (shortest path, non-negative weights)

Use when: shortest path in a weighted graph, all weights >= 0.

```
vector<long long> dist(n, LLONG_MAX);
priority_queue<pair<long long,int>, vector<pair<long long,int>>, greater<>> pq;
dist[src] = 0; pq.push({0, src});
while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (d > dist[u]) continue;  // outdated entry, skip
    for (auto [v, w] : wadj[u]) {
        if (dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w; pq.push({dist[v], v});
        }
    }
}
// dist[v] = shortest distance from src, LLONG_MAX if unreachable
```

===== DSU: COUNT NUMBER OF DISTINCT GROUPS =====

```
set<int> roots;
for (int i=0;i<n;i++) roots.insert(find(i));
int numGroups = roots.size();
```

===== DIJKSTRA: READ WEIGHTED GRAPH INPUT =====

```
vector<vector<pair<int,int>>> wadj(n);
for (int i=0;i<m;i++) {
    int u,v,w; cin>>u>>v>>w; u--; v--;
    wadj[u].push_back({v,w});
    wadj[v].push_back({u,w});   // omit 2nd line if directed
}
```

===== DSU UNION =====

```
// if(find(a)!=find(b)) unite(a,b);
```